Deepak Kharol

Staff Software Engineer · Backend & Distributed Systems
Mumbai, India · contact@deepakkharol.com · linkedin.com/in/deepakharol

Summary

Staff engineer with 4+ years building the message-delivery and real-time decisioning core of CleverTap, a customer-engagement platform that sends billions of messages a month for 2,000+ brands: a Java/MongoDB/Redis pipeline of independently deployed services (ingestion gateway, event processor, sharded in-memory store of user behaviour, and a delivery service that decides, per user and per message, what to send). I own features end-to-end in that pipeline, from design note through hot-path code, flagged rollout and on-call, and lead workstreams spanning several services and teams. I build features as extensible foundations (configuration over constants, pluggable hooks, versioned formats with headroom) that the next feature reuses instead of rewriting. My work runs in production for Flipkart, Nykaa, Kotak Mahindra Bank, Axis Bank and StockX.

Highlights

- Designed the per-user frequency-capping engine that enforces rule-driven message budgets in memory on the send path with no database call; live for 60+ enterprise accounts within six months (Kotak Mahindra Bank group, Axis Bank, Nykaa, Bajaj Finserv, Angel One) and the base of four later products.
- Technical lead for a five-service, event-driven price-drop and back-in-stock trigger for Flipkart (500M+ registered users, 150M-product catalog, ~50M catalog updates a day), built for Big Billion Days scale as 17 deploy-order-safe PRs.
- Extended the AI campaign-ranking system (reinforcement-learning agents optimising reward functions) from choosing the campaign to choosing the content variant each user receives; sold as the Campaign and Variant Decisioning Agents add-on.
- Removed the platform's largest source of slow database queries with a reusable, configurable archival framework, applied first to historical campaign data.

Experience

CleverTap

Mumbai, India

Staff Software Engineer — *Jul 2026 – Present*

Price-drop and back-in-stock trigger (Flipkart) — *event-driven re-engagement across five services*

- Led the design of a pipeline that alerts a shopper when a product they viewed drops in price or returns to stock, for a 150M-product catalog taking ~50M updates a day. Catalog ingestion captures the change, a Redis queue carries it to the event processor, which fans it out to all 47 shards of the user-behaviour store; the delivery service then streams the matching users and enqueues personalised pushes. Back-in-stock alerts ride on the same pipeline.
- Built the delivery side for sale-scale volume: a streaming reader that never buffers a response, a bounded window of concurrent requests, disk-spilling k-way-merge de-duplication, and a pacer that backs off against a queue shared with other tenants' transactional traffic.
- Added the safety rails: freshness and readiness gates that fail closed, all-or-nothing retries per trigger, at-least-once hand-off de-duplicated on a trigger id, out-of-stock withholding, purchase suppression, and a QA runbook with per-stage trace metrics.
- Kept every knob out of code: thresholds, price bands, freshness slack and pacing are per-account configuration re-read each round, so they change mid-sale without a deploy; capture is a pluggable hook any ingestion path can supply.
- Split a 20k-line change into 17 stacked PRs ordered reader-before-writer so a mixed fleet of old and new services is always a valid state; wrote the contract document the platform team's store was built against.

AI campaign ranking and 1:1 individualisation — *reinforcement-learning agents choose the message each user receives*

- The platform's decisioning layer runs reinforcement-learning agents, each bound to a reward function (clicks, conversions, revenue). Nightly, an agent scores every campaign a user qualifies for, and the delivery service sends the top-ranked message that still fits the user's frequency budget: a personalised choice per user. First deployments with Axis Bank and Nykaa (16M+ annual transacting customers); sold as a paid add-on on the top plan.
- Led a six-PR stacked rollout extending the ranker from choosing the campaign to choosing the content variant per user: the agent scores every variant and the delivery service sends that user's winner, not one A/B winner for everyone.
- Designed the versioned 11-byte ranking-store entry (variant id in one nibble, the other reserved so a later field ships without a format bump; older entries still readable), the cross-service contract, and a fail-open resolver whose fallbacks are counted in dispatch diagnostics rather than dropped silently.

Message spacing with deferral — *queue a blocked message instead of dropping it*

- Designed the runtime that, when a per-user time gap blocks a message, computes the next eligible minute (gap clear, quiet-hours shift, delivery-window or end-of-day cutoff), reserves it in the user's budget and re-enqueues through a timed queue. Exposed as an opt-in for both dashboard and API campaigns.
- Made scheduled campaigns survive outages of the user-behaviour store with exponential-backoff retry instead of a terminal stop, after two incidents had permanently killed customer campaigns; retry budget and backoff are a typed per-account configuration with safe defaults.

Senior Software Engineer — *Jan 2024 – Jun 2026*

Frequency capping — *rule-based per-user message budgets, evaluated in memory at send time*

- Designed and built a credit-based engine replacing fixed per-channel counters with a rule model: channel, label, segment and team are optional criteria and daily, weekly, monthly and per-delivery-slot windows are plug-in limit types, so new rule kinds need no dispatch-path change. Shipped as a seven-PR stacked series: storage, dashboard CRUD and REST APIs, rule cache, credit engine, real-time rule propagation to every shard, and a live debug endpoint.
- Modelled each user's state as a compact byte array (3 bytes per rule, 5 bytes per campaign) rebuilt nightly from 30 days of send history, so a cap check on the dispatch hot path is a few array reads with no database call.
- Hardened for scale: 64-stripe locking against cache stampedes, stale-on-error reads, rolling windows, lazy slot resets and an idempotent migration from legacy caps; root-caused a production incident where an unanchored rule window blocked a large media account's morning campaigns.
- Built the read-only budget simulator the AI campaign ranker uses to keep its chosen message within each user's caps. Live for 60+ enterprise accounts within six months of private beta, including Kotak Mahindra Bank group (50M+ customers), Axis Bank (~59M customers), Nykaa, Bajaj Finserv, Angel One, Apollo, Carousell and Cars24.

Campaign data archival — *keep the primary MongoDB cluster fast as campaign volume grows*

- Designed and rolled out a reusable collection-archival framework, applied first to campaign data: finished campaigns older than a configurable age are moved with their statistics to a separate low-index MongoDB cluster (threshold, batch size and schedule all configuration), with transparent archive fallback on read, blocked re-publishing and clone support.

Server-side unified inbox — *Redis-backed in-app inbox channel built for StockX*

- Owned the backend of a new channel that targets users rather than devices: CRUD, dispatch for scheduled, live-trigger and API-trigger campaigns, reporting, billing integration, version history and Redis-outage retry. Wrote the design note and operations handover; supported a $1M ARR customer migration.

Reliability, performance and security

- Fixed a 4.7 GB heap out-of-memory on the dispatch path by replacing a non-thread-safe LRU cache with a lock-free one; made post-publish audit writes asynchronous to cut publish latency; fixed statistics corruption for A/B variants named with dots.
- Completed platform-wide PII encryption in the send path (personalisation, test sends, audience estimates) without affecting dispatch throughput, for regulated clients such as Kotak Mahindra Bank group, Axis Bank, Flipkart and EU-region accounts under GDPR.

Leadership

- Point of contact for a 10-engineer team; mentored three engineers who shipped five production features in two months; on-call owner for the delivery pipeline; company-wide hackathon winner (2025).

Software Engineer *Jul 2022 – Dec 2023*

- **Campaign creation API.** Brought the programmatic API to parity with the dashboard for push, email and SMS (audience, schedule, conversion tracking, priorities) as a per-section handler pipeline; it now creates 40%+ of all campaigns, and v2 audience caps, personalisation validation and payload compression later plugged in as handlers.
- **Send-time content personalisation.** Architected fetching content from client APIs at send time, with Liquid templating of URL parameters and JSON responses, so brands personalise without first syncing data in. Now in GA.
- **Analytics, integrations and security.** Built push render-rate measurement; added phone-number matching to the Facebook and Google Ads audience integrations; remediated penetration-test findings (label and CSV injection).

Backend Intern — built the first campaign API extensions (audience, schedule, setup sections) *Dec 2021 – Jun 2022*

Dell Technologies, Business Intelligence Intern — Power BI and Tableau dashboards *May 2021 – Jul 2021*

Engineering Practices

- **Extensible by design:** behaviour in per-account configuration and flags, not constants; pluggable hooks and handlers at integration points; versioned formats with headroom; each feature built as a foundation the next reuses.
- **Safe change at scale:** every cross-service change is additive and tolerant of a mixed fleet; large features ship as stacked, individually reviewable PRs behind per-account feature flags.
- **Operability first:** per-stage counters, capped trace logs, live debug endpoints for in-memory state, and runbooks written before rollout.
- **Testing:** large unit suites per feature, integration tests against real servers, mutation-checked regression tests per bug.

Technical Skills

Languages	Java 17, Python, C++, SQL
Platform	MongoDB (sharded), Redis, RocksDB, PostgreSQL, OpenSearch · Guice, Vert.x, Spring, JUnit, Mockito, Maven · Docker, AWS, Prometheus/Grafana, Splunk, LaunchDarkly, GitHub Actions
Design	Event-driven pipelines, sharded in-memory state, streaming with back-pressure, idempotent and at-least-once delivery, versioned wire formats, feature-flagged stacked rollouts, multi-tenant isolation

Education and Awards

KIIT, Bhubaneswar — B.Tech in Computer Science, CGPA 9.36 (2018–2022) · Hackathon winner (2025) · Merit Scholarship